

Porównanie listy 3 (klasyfikacji chorób serca) z profesjonalnymi notatnikami Kaggle

1 Wstęp i uzasadnienie wyboru notatników referencyjnych

Poniższy raport przedstawia zestawienie mojego podejścia (tego z początku semestru) co do problemu klasyfikacji, inżynierii danych, wyglądu notatnika z rozwiązaniami zaprezentowanymi przez (mam nadzieję) doświadczonych analityków danych na platformie Kaggle.

Do porównania wykorzystałam następujące notatniki:

- Predicting Heart Disease Using Ensemble Vote (autor: Nikhil Kohli)
- Heart Disease Prediction - Feature Engineering (autor: Housseem Aayed)
- Heart Disease Prediction - Playground Series S6E2 (autor: Sarcasmos)
- Listen to Your Heart - Disease Prediction (autor: Caesar Mario)
- Omówienia rozwiązań zwycięzców z serii Playground S6E2.

Uzasadnienie wyboru: Wybór powyższych prac podyktowany jest ich specjalizacją w poszczególnych etapach analizy. Notatnik Nikhila Kohli został przeze mnie wybrany ze względu na doskonałą, niezwykle czytelną eksploracyjną analizę danych (EDA). Prace H. Aayed i C. Mario świetnie ilustrują niezbędny preprocessing (np. skalowanie), z kolei rozwiązania zwycięzców serii Playground stanowią idealny punkt odniesienia dla zaawansowanych technik uczenia modelu i optymalizacji hiperparametrów.

2 Eksploracyjna Analiza Danych (EDA) i Wizualizacje

Moje podejście: Na początku bardziej skupiałam się na wizualizacji danych bez głębszego wejścia w niej. Myślałam sobie "o fajne wykresy", oraz że im więcej tym lepiej. Jak się można domyślić to nie jest prawda i często mniej oznacza lepiej. W laboratoriach mam dużo nieopisanych wykresów, a nie do końca polega na tym EDA. Zrobiłam standardowe rzeczy (zbadanie duplikatów, sprawdzenie brakujących wartości) co jest jak najbardziej dobrą praktyką. Zauważyłam, że bardzo lubię histploty i countploty.

```
In [4]: df_heart = df_heart.rename(columns= {'cp':'chest_pain_type', 'trestbps':'resting_BP', 'cho
1':'serum_cholesterol', 'fbs':'fasting_blood_sugar', 'restecg':'resting_ECG',
                                           'thalach':'max_heart_rate', 'exang':'exercise_induced_an
gina',
                                           'ca':'major_vessels_count', 'thal':'thallium_stress'})

df_heart.columns
```

Feedback na podstawie notatników z Kaggle: Feedback dotyczący EDA będę bazować na notatniku: *Predicting Heart Disease Using Ensemble Vote* (Nikhil Kohli). Dobrą praktyką dla mnie może być zmiana też nazw kolumn, tzn. zamiast randomowych literek pełną nazwą cechy. Ułatwiłoby to znacznie czytanie wykresów.

Bardzo spodobał mi się sposób na automatyczną selekcję, które kolumny mają wartości dyskretne, a które ciągłe.

Categorical Discrete & Continuous Variables

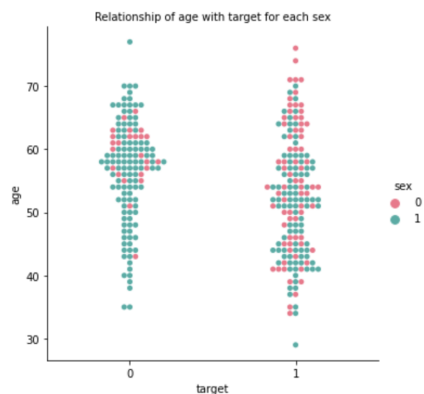
```
In [7]: categorical_cols = []
        continuous_cols = []

        for column in df_heart.columns:
            if len(df_heart[column].unique()) <= 10:
                categorical_cols.append(column)
            else:
                continuous_cols.append(column)
```

Dodatkowo dowiedziałam się, że bardzo czytelne są wykresy typu swarmplot, gdzie widać jak wygląda target względem wartości czegoś (mega przydatne w danych medycznych, bo widać, że jak cholesterol powyżej danej wartości to pewnie będzie źle).

```
In [27]: for index, column in enumerate(continuous_cols):
        plt.figure(index, figsize=(7,5))
        sns.catplot(x='target', y=column, hue='sex', kind='swarm', data=df_heart, palette='hus
1')
        plt.title('Relationship of {} with target for each sex'.format(column), fontsize = 10)
```

<Figure size 504x360 with 0 Axes>



Dodatkowo dobrze by było jakbym pisała więcej wniosków, mniej wykresów.

3 Przetwarzanie Danych (Preprocessing)

Moje podejście: Nie wygląda on zbyt obszernie. Mamy wcześniej sprawdzone braki w danych oraz duplikaty i zwykły one-hot-encoding do danych kategoriowych (`pd.get_dummies`). Następnie mam podział na zbiór testowy i treningowy i w sumie na tyle by było.

Feedback na podstawie notatników z Kaggle: Z analizy takich źródeł jak *Heart Disease Prediction - Feature Engineering* (Houssem Aayed) oraz *Listen to Your Heart* (Caesar Mario) wynika, że brak pustych wartości rzadko oznacza gotowość zbioru do treningu. Autorzy zawsze aplikują dedykowane przekształcenia. Po pierwsze, zmienne kategoriowe (np. *cp* czy *thal*) poddawane są kodowaniu *One-Hot Encoding*. Bez tego algorytmy mogłyby błędnie uznać kategorię „3” za silniejszą niż „1”. Po drugie, standardem (szczególnie w notatniku C. Mario) jest użycie *StandardScaler* do ujednolicenia rzędów wielkości zmiennych ciągłych (np. cholesterolu), co gwarantuje poprawną zbieżność modelu. U mnie zabrakło jakiegokolwiek ujednolicenia danych (żadnego *minmaxa* ani *standardscaler* np.) co najwidoczniej jest przydatne w tym datasetcie.

Wnioski: Ogólnie identyfikacja czystości zbioru to jest kluczowy krok, ale jednak w większości przypadków nie ma idealnego przepisu na preprocessing danych, więc ciężko mi skonstruować jakiś sensowny feedback dla mnie. Wiem, że w liście 3 mogłam zrobić jakieś skalowanie danych co mogłoby wyjść dobrze dla LogReg, a dla ogólnej zasady dobrze jest poznać swoje dane.

4 Optymalizacja Hiperparametrów

Moje podejście: Cała lista bardzo mocno opierała się na badaniu hiperparametrów, wpływu stratyfikacji oraz CV dla różnej liczby podziałów. Jest to dobra praktyka, ponieważ zapobiega przetrenowaniu i uczy mnie diagnozowania skuteczności modelu oraz tego jak można go dostroić.

Feedback na podstawie notatników z Kaggle: Platforma Kaggle rządzi się prawami maksymalizacji wyników, co idealnie obrazują opisy rozwiązań zwycięzców pierwszych miejsc z edycji *Playground S6E2*. Zwycięzcy często do testowania hiperparametrów tworzą dzienniki i po prostu trenują te kilkanaście kombinacji.

Wnioski: Tutaj moje podejście znacząco się nie różni, różni się tylko implementacja. Myślę, że to jest dobra praktyka, zwłaszcza na początku nauki.

5 Uczenie Modelu

Moje podejście: Moje uczenie modelu wygląda bardzo basicowo, nic wielkiego, tuning głównie działy się w dalszych sekcjach w ramach rozwiązania zadań z listy, co będzie ogólną dobrą praktyką dla mnie w przyszłości.

```
# Simplified hyperparameter grids
logit_params = {"model__C": [0.1, 1.0]} # Removed penalty to reduce combinations

hgb_params = {
    "model__learning_rate": [0.1], # Single value for speed
    "model__max_depth": [3],
}

# Logistic Regression tuning
print("Fast-tuning Logistic Regression...")
logit_search = RandomizedSearchCV(
    logit_pipe,
    logit_params,
    n_iter=1, # Minimum iterations
    scoring="roc_auc",
    cv=2,
    n_jobs=-1,
    random_state=42,
    verbose=0 # Reduced verbosity
)
logit_search.fit(X_train, y_train)
```

```
In [15]: # --- Logistic Regression Parameters ---
parameter_lr = {"algo__solver": ["lbfgs", "saga", "newton-cg"],
                , "algo__C": [0.1, 0.2, 0.5, 0.8]}

# --- Logistic Regression Algorithm ---
algo_lr = LogisticRegression(penalty="l2", random_state=42, n_jobs=-1)

# --- Applying Logistic Regression ---
acc_score_train_lr, acc_score_test_lr, best_score_lr = fit_ml_models(algo_lr, parameter_lr,
"Logistic Regression")

... Fitting Logistic Regression ...
Fitting 10 folds for each of 12 candidates, totalling 120 fits

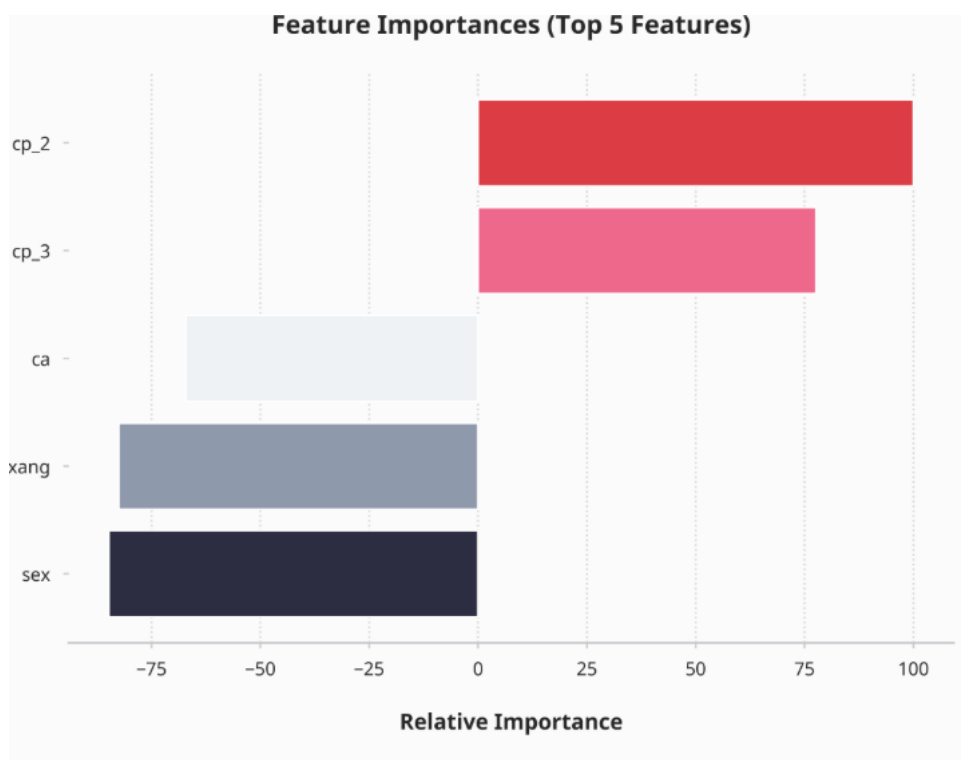
>> Best Parameters: {'algo__C': 0.5, 'algo__solver': 'lbfgs'}
>> Best Score: 0.823
```

Feedback na podstawie notatników z Kaggle: Notatnik: *Listen to Your Heart* oprócz tego, że jest bardzo estetyczny (też uwaga dla mnie) ma fajny wykres przy ewaluacji LogReg co może pomóc w dalszym tuningowaniu:

6 Ewaluacja Klasyfikatora

Moje podejście: Do oceny jakości klasyfikacji użyłam metryk, które są bardzo ważne w medycynie. Nie kieruję się tylko zwykłym accuracy, tylko także recall'em, macierzą pomyłek.

Feedback na podstawie notatników z Kaggle: Profesjonaliści zajmujący się medycznymi zbiorami danych kładą specyficzny nacisk na wybrane metryki ze względów etyczno-biznesowych. Na przykład w notatniku *Heart Disease Prediction* (Sarcasmos) autorka/autor bezpośrednio wskazuje, że przy klasyfikacji chorób wskaźnik Czułości (*Recall*)



musi być najwyższym priorytetem. Sarcasmos oraz inni uczestnicy często modyfikują próg odcięcia prawdopodobieństwa (*threshold tuning*), aby zmaksymalizować czułość modelu, nawet kosztem precyzji.

Wnioski: Mój zestaw użytych metryk jest w pełni kompletny i poprawny. Jednak raport znacząco by zyskał (medycyna także), jakbym ustaliła wagę czułości, nadając priorytet błędom False Negative podczas oceny modelu.

7 Podsumowanie i odpowiedzi na pytania

Podjęście studenckie to solidny punkt wyjścia, realizujący kluczowe dla inżynierii danych założenia badawcze i ewaluacyjne (stratyfikacja, różne warianty CV). Konfrontacja tego kodu ze specyfiką Kaggle dowodzi jednak, że praca zawodowa silnie integruje obligatoryjny preprocessing zmiennych z użyciem *OHE* i skalerów (C. Mario, H. Aayed) oraz świadomą, pro-medyczną interpretację metryk stawiającą *Recall* na pierwszym miejscu.

Odpowiedzi na pytania pomocnicze:

- **Czym różnią się poszczególne etapy?** W mojej pracy etapy EDA i preprocessingu mają na celu zapoznanie się z danymi i spełnienie minimum wymagań do uruchomienia modelu. Na Kaggle są to procesy silnie nastawione na inżynierię (Feature Engineering), gdzie każda wykryta zależność przekłada się na stworzenie nowych cech (np. grupowanie wieku). Samo uczenie modeli w listach laboratoryjnych to zazwyczaj aplikacja jednego algorytmu, podczas gdy profesjonaliści budują złożone zespoły modeli (Ensembles).
- **Które czynniki najsilniej wpływają na osiąganе wyniki?** Z analizy prac konkursowych jasno wynika, że największy wpływ na poprawę wyników ma wysokiej

jakości *preprocessing* i dobór cech, a w kwestii samego modelu – rygorystyczna i automatyczna optymalizacja hiperparametrów oraz dobór odpowiedniego progu predykcji pod konkretną metrykę (np. Recall).

- **W jaki sposób wskazane metody można by zastosować w liście z zajęć?**
Własny kod z Listy 3 można w prosty sposób ulepszyć wdrażając *StandardScaler* do wyrównania wag zmiennych, używając wykresów wielowymiarowych (np. *swarm-plot*) do budowania głębszych wniosków w sekcji EDA oraz w mądrzejszy sposób dokonać wyboru hiperparametrów (ponoć są jakieś biblioteki do tego).